Highlights

From detection to action: closing the autonomic loop of a self-coordinated manufacturing line with rule-based and retrieval-augmented LLM planners

Manuel Sánchez, Ernesto Exposito

- The 2020 autonomic architecture detected conditions but never planned or acted.
- Its alarms are sparse line-wide bursts; with them no planner recovers delay.
- A per-station detector on processing times makes closed-loop repair worthwhile.
- A frontier LLM vetoing a statistical test's candidates doubles the best rule.
- A fixed fault onset inside the alarm burst had produced misleading gains.

From detection to action: closing the autonomic loop of a self-coordinated manufacturing line with rule-based and retrieval-augmented LLM planners

Manuel Sánchez[a,b,*], Ernesto Exposito[b]

[a] *Universidad Nacional Experimental del Táchira, San Cristóbal, Venezuela*

[b] *Univ Pau & Pays Adour / E2S UPPA, Laboratoire d'informatique de l'Université de Pau et des Pays de l'Adour (LIUPPA), EA3000, 64000 Pau, France*

Abstract

Autonomic architectures for Industry 4.0 promise self-healing production, but published systems often stop at detection. We revisit our own 2020 self-coordinated manufacturing architecture, find that it implements Monitor and Analyze but not Plan, Execute or a Knowledge base, port its simulator to a tested headless engine over the Bosch production-line dataset, and close the loop with an action space built on the line's parallel twin stations, a symbolic validator, synthetic step, intermittent and drift faults, and an explicit effect model. We compare rule-based planners with large-language-model (LLM) planners, with and without retrieved operational knowledge, on a local 7B model (Qwen2.5) and a frontier model (Claude Fable 5.1), selecting every design choice on a development trace and evaluating on 216 fault scenarios from three held-out traces, by the delay recovered within 24 h of the fault. The original Analyze layer is the bottleneck: within a day of a fault it names the faulty station in only half of the scenarios, a few times and always inside line-wide bursts of alarms, and with it no planner recovers more than 7% of the delay. A per-station detector on processing times, chosen on the development trace, lets the best rule recover 20%. With that detector, a frontier LLM that vetoes the candidates of a statistical change test recovers 41% (95% interval 32–50%), ahead of every rule configuration we tried, including one chosen on the held-out traces, and of the same design on the 7B model (29%); used without the test, the same model does

*Corresponding author.

Email addresses: mbsanchez@unet.edu.ve (Manuel Sánchez), ernesto.exposito@univ-pau.fr (Ernesto Exposito)

not beat the rule. A first evaluation design, with the fault injected at a fixed time inside the detector's start-up burst, had produced apparent gains that reversed this ranking.

Keywords: Autonomic computing, MAPE-K, Industry 4.0, Large language models, Retrieval-augmented generation, Fault management

---

1. Introduction

Autonomic computing proposes systems that manage themselves through a Monitor–Analyze–Plan–Execute loop over a shared Knowledge base (MAPE-K) [1, 2]. In manufacturing, this loop is attractive because production lines degrade continuously and human supervision does not scale: a self-healing line would notice a degrading station and reconfigure itself before the delay accumulates.

In 2020 we proposed an architecture for self-coordinated manufacturing processes with self-* properties and evaluated it with a simulator, SSACSimulator, over the Bosch production-line dataset [3, 4]. Revisiting that code six years later, we found that it implements the Monitor and Analyze phases only. When one of its three detectors fires, the simulator writes *"launching the autonomous cycle for reconfiguration"* to a log panel; no plan is produced and nothing is executed. The loop was announced, not closed.

This paper closes it, and asks what closing it actually requires. Two developments make the question timely. First, large language models (LLMs) are increasingly proposed as planners for industrial decision making, from the control of modular production systems [5] to advanced planning and scheduling [6]. Second, retrieval-augmented generation (RAG) [7] offers a natural candidate for the Knowledge component of MAPE-K: instead of a hand-built ontology, the planner retrieves the operational history it needs. Whether either helps, against which baseline, and under which Analyze layer, is an empirical question.

Our contributions are:

1. A faithful, tested, headless port of the 2020 simulator, and an analysis of its Analyze layer on the published data: one detector is unreachable, one is exercised by 0.004% of the events, and the third raises line-wide bursts of alarms, concentrated at the start of each trace, that name a faulty station within a day in only half of the scenarios (Sections 3 and 7.1).
2. The missing Plan, Execute and Knowledge phases, and a per-station Analyze on processing times: an action space based on the parallel

"twin" stations of the Bosch line, a symbolic validator, fault injection with three fault shapes, and an explicit, parameterised effect model (Section 4).

3. A comparison of rule-based planners against LLM planners with and without retrieved knowledge, and of a hybrid in which a statistical test proposes and the LLM vetoes, on a local 7B model and a frontier model, under both Analyze layers, with every design choice made on a development trace and 216 fault scenarios on three held-out traces (Sections 5–7).
4. Evidence that the Analyze layer, not the planner, decides whether closing the loop recovers production; that, given useful alarms, a capable LLM adds most as a veto over a statistical test; and that a fixed fault onset can manufacture conclusions that reverse under a sounder design (Section 8).

All code, data-processing scripts, prompts, every model decision with its stated reason, and all results are available (Section Appendix A).

2. Background and related work

Autonomic computing and self-adaptive systems. The MAPE-K reference model [1] structures a self-managing system as a feedback loop around a managed resource. Engineering such loops is the subject of the self-adaptive systems literature [2], which stresses that the Analyze phase must separate genuine change from noise before any adaptation is planned. LLMs have recently been examined as components of such loops: a state-of-the-art review and research roadmap organises their potential uses around the MAPE-K functions [8, 9]; LLM-based agents have been embedded in MAPE-K-based multi-agent systems [10]; and MAPER extends MAPE-K with an LLM-based reasoning component to handle unanticipated runtime situations [11]. Our study differs in placing an LLM inside the loop of an industrial system whose detector we first inherit unchanged and then replace, and in measuring it against tuned non-LLM baselines on held-out data.

Self- properties in manufacturing.* Our earlier work integrated multi-agent systems, service orientation and autonomic computing for Industry 4.0 process coordination [3, 4, 12]. The present paper re-examines that architecture empirically rather than proposing a new one.

Alarm management and change detection. In process industries, alarm management standards require alarms to be rationalised, so that each one is meaningful and calls for a response, before operators are asked to act on them [13]. An autonomic controller that acts on alarms faces the same requirement, without an operator to absorb the noise. Detecting a change in the rate of an event stream or in the distribution of a measurement is a classical problem [14]; our rule-based planners and our per-station detector are simple instances of it.

LLMs and retrieval in industrial decision making. RAG conditions a language model on documents retrieved for each query [7]. In manufacturing, LLM agents have been combined with digital twins to plan and control modular production [5] and applied to advanced planning and scheduling [6], and a recent review of LLMs for fault diagnosis in smart manufacturing covers, among other roles, retrieval-augmented fusion of industrial knowledge [15]. Here, the retrieved knowledge is structured operational statistics computed by code from the controller's own history, not documents; we keep the term RAG for the retrieval step and state exactly what is retrieved (Section 5).

3. The 2020 system revisited

3.1. Simulator and data

SSACSimulator is a Qt/C++ discrete-event simulator. A process model, derived from the Bosch Production Line Performance dataset [16], describes 52 stations on four sub-lines plus begin and end nodes, 285 transitions (84 of them main flows), a throughput time of 6 420 min and 1 183 165 cases. Four event traces of 100 000 transitions each replay recorded piece movements; they span 54 to 168 hours of production. Production is recorded in batches: 62 of the 157 hours of trace 100m–200m contain no event.

3.2. Headless port

We ported the simulation core (about 1 440 lines of C++ logic, excluding the graphical user interface) to Python. The port reproduces the original's behaviour as read from its source, including details that change results: thresholds computed in single precision, a 100 000-event loading limit, and a lenient rule for malformed rows. A fast path advances only the pieces that can leave the system; the original per-tick loop is kept as a test oracle, and a regression test checks that both produce identical detections, decisions and piece lifetimes. Parity is with the original as read from its source, not with the compiled Qt binary. A run takes 0.2–0.4 s.

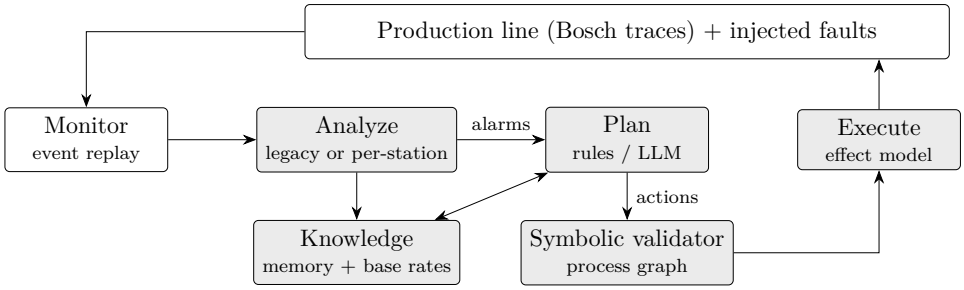


Figure 1: The closed MAPE-K loop. White: present in the 2020 system. Grey: added or replaced in this work. The Analyze layer is either the legacy detector or the per-station detector of Section 4.4.

3.3. What the Analyze layer actually does

The original implements three detectors: a station whose mean transition time exceeds 1.3 times its reference; more than 10% of stations flagged within one batch of simultaneous events; and a mean piece lifetime exceeding 1.1 times the throughput time. Only the second fires. The predicted-failure detector depends on a classifier label that is positive in 15 of 400 000 events across all traces (0.004%). The global detector is unreachable: the mean piece lifetime (636 min) is an order of magnitude below its threshold (7 062 min). Finally, 45 of the 285 transitions have a reference time of zero, so any positive duration flags their station. Section 7.1 shows what the one active detector sees once faults are injected.

Plan, Execute and Knowledge are absent. This is the starting point for the rest of the paper.

4. Closing the loop

Figure 1 summarises the closed loop. Components added in this work are shaded.

4.1. Twin stations and the action space

The Bosch line is built from parallel station pairs (and a few triples) that share predecessors and successors and split the load almost evenly; for example, on their main flows stations L0_S2 and L0_S3 receive 339 345 and 334 054 cases. We identify twins automatically from the overlap of their predecessor and successor sets on main flows: 29 of the 52 stations have one. Diverting a piece from a station to its twin keeps its trajectory consistent with the process graph. Structural equivalence does not guarantee

equivalent equipment or capacity, which the anonymised dataset does not describe; a plant would have to confirm each twin. The action space is REROUTE(S, T, f), which diverts a fraction f of the pieces entering S to its twin T; RESTORE(S); MAINTENANCE(S, d), which repairs S for d minutes; and NOOP.

4.2. Symbolic validator

Every action, from any planner, is checked against the process graph and the current interventions before execution: stations must exist, a reroute target must be a structural twin, reroutes must not form chains, and a station cannot be repaired twice concurrently. The validator never judges whether an action is a good idea, only whether it is admissible.

4.3. Effect model

The simulation is trace-driven: start times come from the recorded data and are not shifted; faults and actions act on transition durations, and the lifetime of a piece is the sum of its transition durations. Delay therefore does not propagate as queueing; the model estimates time lost per piece. A Bosch "transition" is the time between two station timestamps and mostly measures waiting: its median is 0 min and its 90th percentile 18–30 min, but its 99th percentile reaches 4 100–5 900 min. Effects are therefore *additive* and proportional to each station's processing time τ_S (the median of its transitions of up to 60 min), never multiplicative over a transition that includes days of storage. τ_S is 6 or 12 min for the eight stations where faults are injected, identical on all traces. The effect model is:

- **Fault** of factor k on S: $+(k(t) - 1)\,\tau_S$ on every transition leaving S, where $k(t)$ is a *step* (k from t_0), *intermittent* (k for 60 min, 1 for 120 min, repeating) or *drift* (rising linearly from 1 to k over 600 min).
- **Reroute**: a deterministic fraction of pieces (selected by a hash of the case identifier) goes to T, which then adds $\alpha\, f\, \tau_T$ for the extra load.
- **Maintenance**: faults on S already active when the repair starts are cleared when it ends; a fault that begins during a repair is not. During the repair, in *contingency* mode (default) backup capacity outside the model processes the pieces, adding $0.5\,\tau_S$; in *stop* mode the pieces wait for the repair to finish. Repairing a healthy station costs and fixes nothing, which is what makes false alarms expensive.

Because the model is additive, the excess delay of a run decomposes exactly into the minutes added to individual transitions, which lets us measure it within any time window. The dataset is anonymised and provides no capacities, repair times or labelled failures, so all faults are synthetic and every parameter we could not ground in data (severity k, repair time d, congestion α, maintenance mode) is varied in a sensitivity analysis (Section 7.2).

4.4. A per-station Analyze on processing times

Section 7.1 shows that the legacy detector reports the injected faults rarely and only inside line-wide bursts. Its station test compares means of whole transitions, dominated by hours or days of storage, so a fault that adds minutes of processing barely moves them. We therefore add an alternative Analyze layer that looks only at the processing regime: the transitions leaving a station that last at most 60 min. For each station, causally, it takes as reference the 75th percentile of those transitions over the previous 24 h and the rate p_0 at which they exceed it (Laplace-smoothed, because timestamps have a 6-min granularity); among the station's last N processing transitions, it counts those above the reference and flags the station when that count is improbable under Binomial(N, p_0) at level α. Nothing is flagged during an 8-h warm-up; the stations flagged in one batch of simultaneous events are reported in a single alarm, and a station is not flagged again within a refractory period. N, α and the refractory period were chosen on the development trace (Section 6): $N = 10$, $\alpha = 10^{-3}$ and 6 min. With this layer, the legacy station alarms are no longer passed to the planners. The planners themselves are unchanged.

5. Planners

All planners receive, for each alarm, the alarm and the state of their own past interventions. None sees the injected faults. All of them respond to a station judged faulty in the same way: reroute to a free twin if one exists, then repair. Every planner, including the LLM planners, leaves a station it has acted on alone for 240 min.

5.1. Rule-based planners

- **Naive**: acts on every station named in an alarm.
- **Calibrated**: acts when the number of alarms on a station within a 120-min window is improbable (Poisson tail, $\alpha = 10^{-3}$, at least 3 alarms) under its base rate, learned on a *different* fault-free trace.

- **Adaptive**: the same test, against the rate observed in the preceding 24 h of the same trace, causally, after an 8 h warm-up.
- **Adaptive + load veto** (control): the adaptive rule, ignoring stations that receive load the controller itself rerouted onto them.
- **Twin-only** (control): acts on every flagged station that has a free twin and ignores the others.

The two controls are not proposed methods; they implement as plain rules behaviours observed in LLM planners, so that an LLM's contribution is measured beyond them. The default window, level and history above were fixed before any evaluation; we also re-tune them on the development trace (Section 6).

5.2. LLM planners

For every alarm with actionable stations, the LLM must return, for each of them, a verdict (*degraded*, *normal*, *self-inflicted* or *insufficient evidence*) followed by an action type (none, repair, or reroute and repair). The output is constrained by a JSON schema whose station names are restricted to the actionable ones, so the decoder cannot invent a station. Code chooses the twin, discards any action inconsistent with the model's own verdict, and passes the rest to the validator.

- **C1** receives the alarm, the actionable stations, whether each has a free twin, and the current interventions: the same structural information as the rules.
- **C2 (RAG)** additionally receives knowledge retrieved from the controller's operational memory: for each station, the alarms observed in the last 2 and 24 h against the number its fault-free base rate predicts, whether the controller is pushing rerouted load onto it, whether its alarms dropped after past repairs, and its most recent log records. Retrieval is keyed by station and ordered by recency; all statistics are computed by code.
- **Hybrid**: the adaptive change test proposes the stations; the LLM, with C2's knowledge, confirms or vetoes and chooses the action. A vetoed station is not re-examined for 120 min. The prompt tells the model that the test is usually right but cannot see context, and gives as examples load the controller rerouted onto a station, a past repair that did not reduce its alarms, and *"a burst of alarms shared by many stations at once"*.

The system prompt describes how the Analyze layer raises alarms. Prompt version v2 describes the legacy detector; v3, used with the per-station detector, differs from v2 only in that paragraph. Both were fixed on the development trace before any held-out run.

Models. We use Qwen2.5-7B-Instruct [17], 4-bit quantised, served locally with Ollama [18] at temperature 0; and Claude Fable 5.1 (Anthropic), accessed through the Claude Agent SDK [19] with effort set to *low*, no tools and no external context sources. The frontier model does not expose a temperature setting, so its runs are not bit-reproducible. Every request and response of both models is stored, keyed by the full request. Identical requests are frequent, because before the fault onset a scenario yields the same states as the fault-free run of its trace, and they are served from this store: with the per-station detector, 2 403 of the 26 933 calls of C2 on Fable 5.1 were new. A new call takes a median of 5–8 s with either model, short against faults that cost hours. Each decision is a single sample of the model; we did not estimate run-to-run variability. Fable 5.1 was used through a Claude subscription, with model identifier `claude-fable-5-1`, between 6 and 15 October 2026. A first development pilot was discarded because the SDK passed the tool definitions of the account's connectors to the model; the harness was then restricted to no tools and its identifier added to the cache key, so that no response from that pilot was reused. The prompts are given in Appendix Appendix A.

6. Experimental protocol

Scenarios. A fault of factor $k = 3$ is injected on one of eight stations, four with a twin (L0_S2, L0_S4, L0_S6, L3_S35) and four without (L0_S1, L0_S8, L3_S29, L3_S33), all of which have observable processing times on every trace, with one of the three fault shapes, at one of three onsets per station and shape. Onsets are drawn uniformly between 24 h after the first event of the trace and 24 h before its last, from a generator seeded by the scenario's name. Each run stops 24 h after the onset. Each trace also has a fault-free run over its whole length. Repairs last 60 min, $\alpha = 0.5$, and maintenance runs in contingency mode unless stated otherwise.

Development and held-out traces. The first trace (0–100m) is the development trace; the other three are held out (216 fault scenarios). On the development trace only, we chose the parameters of the per-station detector (27 configurations, selected by the recovery of the adaptive rule with load veto),

re-tuned the rules (45 configurations of window, level and history for each adaptive rule, 15 of window and level for the calibrated rule), and fixed the prompts. Every prompt version tried and its results are logged in the repository. Calibrated rules and C2 learn their base rates on a fault-free trace other than the one they are evaluated on (100m–200m from the development trace; 200m–300m and 500m–600m from each other). During prompt development, the development runs took their base rates from 100m–200m, which is later held out; this exposes no fault labels.

Metrics. Every run replays the same trace, so each piece is compared with itself in a fault-free reference run. The *24-h recovery* of a planner over a set of scenarios is $1 - \sum E_{24}(\text{planner}) / \sum E_{24}(\text{no action})$, where E_{24} is the delay added to transitions in the 24 h after the fault onset: 1 erases the cost of the faults; 0 is no better than doing nothing; negative values mean that the planner's own actions cost more than the faults. A planner *acts* on a scenario if it repairs the faulty station within those 24 h; *false repairs* are repairs of other stations in the same window. The *false-alarm cost* is the excess delay per hour in the fault-free runs.

Statistics. The primary estimand is the difference in 24-h recovery between two planners, with a 95% bootstrap interval [20] that resamples scenarios within their trace (10 000 resamples). Because scenarios on the same trace share their alarm noise and three traces are few clusters, we also report a two-level bootstrap that resamples traces and then scenarios, and the difference with each trace left out. Secondary: the Wilcoxon signed-rank test on per-scenario delay [21], which treats scenarios as independent and is therefore conditional on these traces, with Holm adjustment [22] over the rows of each table block and the matched-pairs rank-biserial correlation r as effect size. The breakdown by fault shape is exploratory.

7. Results

7.1. The legacy Analyze layer gives too little evidence

The legacy station detector raises its alarms in bursts that follow the production batches rather than the faults. Without faults, on the held-out trace 100m–200m it raises 63, 18, 0, 0, 0, 2, 0 and 1 alarms in successive 12-h blocks; on 200m–300m, 66 in the first 12 h, none in the next 60 h, then 25, 18 and 16; on the development trace, 30, 9 and then none. Table 1 shows what it reports in the 24 h after a fault, with no planner. It names the faulty station in half of the held-out scenarios, 5.4 times on average against

Table 1: What each Analyze layer reports about the faulty station in the 24 h after the fault onset, with no planner, on the 216 held-out fault scenarios, and in the same windows without the fault.

Analyze layer	Scenarios with an alarm (fault / no fault)	Alarms per scenario (fault / no fault)	Stations per alarm
Legacy detector	108 / 52	5.4 / 1.2	6.5
Per-station detector	122 / 107	15.1 / 3.6	2.3

1.2 in the same window without the fault, and each such alarm lists 6.5 stations on average, because the detector only fires when more than 10% of the stations are flagged in one batch. It compares mean transition times that are dominated by storage, so a fault that adds 12 min of processing moves them little.

With this Analyze layer, no planner recovers delay within 24 h (Table 3, upper block): the rules range from -0.03 to -0.02, C2 on Fable 5.1 reaches $+0.07$ $[-0.00, +0.14]$, and the hybrid on Fable 5.1, which vetoes almost every candidate, $+0.03$. The planners have too little to act on.

7.2. A per-station Analyze layer

The per-station detector is noisier in absolute terms (Table 1): it names a station that has no fault in half of the windows. But it reports the faulty station three times as often as the legacy detector, 15.1 alarms per scenario against 3.6 without the fault, and in alarms that list 2.3 stations on average, so a rate test on its alarms can single the station out. On the development trace, every configuration of the per-station detector names the faulty station within 24 h in all 72 development scenarios; the chosen one lets the adaptive rule with load veto recover $+0.84$ there. Table 2 reports the rule-based planners with this detector on the held-out traces and a one-at-a-time sensitivity analysis over all fault shapes. Acting on the detector's alarms pays off only when faults are severe relative to the cost of a repair and backup capacity absorbs the work during repairs. Halving the fault factor ($k = 1.5$), quadrupling the repair time (240 min) or stopping the line during repairs makes every rule much worse than doing nothing, because the false repairs then cost more than the faults they fix; full congestion ($\alpha = 1$) cancels the benefit. Shorter repairs, more severe faults or no congestion more than double it. The naive planner, which acts on every alarm, loses against doing nothing at the central point, and intermittent faults defeat every rule there. This analysis covers the rules only; we did not repeat it for the LLM planners.

Table 2: Rule-based planners with the per-station detector, held-out traces (216 fault scenarios): 24-h recovery. Central point: $k = 3$, 60 min repairs, $\alpha = 0.5$, contingency mode. Sensitivity rows: all fault shapes, one parameter changed. Default hyperparameters.

	Naive	Calibrated	Adaptive	Twin-only
All fault shapes	−0.28	+0.13	+0.17	−0.20
step	−0.07	+0.23	+0.23	−0.07
intermittent	−1.12	−0.29	−0.27	−0.64
drift	−0.10	+0.22	+0.30	−0.13
$\alpha = 0$	–	+0.37	+0.41	+0.22
$\alpha = 1$	–	−0.07	−0.03	−0.61
$k = 1.5$	–	−1.51	−1.47	−1.75
$k = 5$	–	+0.37	+0.41	+0.03
Repair 15 min	–	+0.54	+0.58	+0.20
Repair 240 min	–	−1.22	−1.22	−1.73
Stop mode	–	−2.48	−2.55	−0.81

7.3. Held-out comparison

Table 3 and Figure 2 report all planners on the three held-out traces. With the per-station detector, the rules recover 13–20% with their default settings; re-tuning on the development trace helps the calibrated rule (+0.27) but not the adaptive ones (+0.14 and +0.18). The naive planner loses against doing nothing under both detectors (−0.25 and −0.28). The development trace is markedly easier than the held-out ones (+0.84 against +0.20 for the same rule).

With the per-station detector, the planners separate. The hybrid on Fable 5.1 reaches +0.41, the highest recovery, while repairing 1.8 healthy stations per scenario and costing 25 piece-min/h without faults, against 5.5 and 89 for the adaptive rule with load veto. It is the only planner with a clearly positive recovery on intermittent faults (+0.29). It acts on the faulty station in fewer scenarios than the rules (87 against 108 of 216) but loses much less to false repairs. The same hybrid on the 7B model reaches +0.29. Used without the test, the LLM planners do worse: C2 on Fable 5.1 reaches +0.14, below the rule, and C2 on the 7B model −0.25, with 14 false repairs per scenario. Without retrieved knowledge, Fable 5.1 (C1) acts on the faulty station in only 19 scenarios and almost never on a healthy one; the 7B model acts anyway. Figure 3 places every planner on the trade-off between recovery under faults and the cost of acting on noise.

Table 3: Held-out traces (216 fault scenarios, 3 fault-free runs), 24-h horizon. Recovery: with 95% bootstrap interval stratified by trace. Acts: scenarios in which the faulty station is repaired within 24 h. False repairs: repairs of other stations within 24 h, per scenario. False-alarm cost: piece-min/h in fault-free runs, mean of the three traces.

Planner	Model	Recovery	Step	Interm.	Drift	Acts	False repairs	False-alarm cost
Legacy detector								
Calibrated rule	–	−0.03 [−0.12, +0.04]	+0.04	−0.38	+0.04	62	4.4	108
Adaptive rule	–	−0.03 [−0.14, +0.06]	+0.06	−0.44	+0.06	77	6.2	67
Adaptive + load veto	–	−0.02 [−0.12, +0.07]	+0.08	−0.40	+0.06	75	5.9	64
C2 (RAG)	Qwen 7B	−0.15 [−0.27, −0.05]	−0.06	−0.73	+0.02	104	9.5	159
Hybrid	Qwen 7B	−0.04 [−0.10, +0.03]	−0.02	−0.25	+0.05	51	4.0	38
C2 (RAG)	Fable 5.1	+0.07 [−0.00, +0.14]	+0.13	−0.17	+0.12	54	3.9	58
Hybrid	Fable 5.1	+0.03 [−0.01, +0.08]	+0.09	−0.03	+0.00	24	0.9	7
Per-station detector								
Calibrated rule	–	+0.13 [+0.01, +0.24]	+0.23	−0.29	+0.22	101	5.5	107
Adaptive rule	–	+0.17 [+0.05, +0.27]	+0.23	−0.27	+0.30	108	5.9	98
Adaptive + load veto	–	+0.20 [+0.09, +0.30]	+0.26	−0.20	+0.33	108	5.5	89
Twin-only (control)	–	−0.20 [−0.30, −0.11]	−0.07	−0.64	−0.13	60	6.2	108
C1	Qwen 7B	−0.05 [−0.17, +0.06]	+0.06	−0.55	+0.06	90	7.6	121
C2 (RAG)	Qwen 7B	−0.25 [−0.42, −0.10]	−0.05	−1.06	−0.08	117	14.0	203
Hybrid	Qwen 7B	+0.29 [+0.18, +0.38]	+0.32	−0.03	+0.41	106	5.2	74
C1	Fable 5.1	+0.08 [+0.04, +0.14]	+0.02	+0.10	+0.14	19	0.1	1
C2 (RAG)	Fable 5.1	+0.14 [+0.02, +0.24]	+0.23	−0.25	+0.22	105	7.1	100
Hybrid	Fable 5.1	**+0.41** [+0.32, +0.50]	+0.35	+0.29	+0.53	87	**1.8**	**25**

7.4. Statistical comparison

Table 4 compares the hybrid on Fable 5.1 with the other planners under the per-station detector. Against the adaptive rule with load veto its recovery is higher by +0.21 [+0.14, +0.28]; it loses less production time in 139 scenarios and more in 18 ($r = 0.78$). The advantage holds on each held-out trace (+0.23, +0.15 and +0.22), with any trace left out (+0.19 to +0.23), and when traces are resampled as clusters ([+0.11, +0.28]; positive in all 27 ordered draws of three traces). It also holds against the rules re-tuned on the development trace (+0.15 [+0.08, +0.22] against the best of them) and even against the configuration of the grid that is best on the held-out traces themselves (+0.13 [+0.06, +0.21]), which no honest protocol could select. It is larger on intermittent (+0.49) and drift faults (+0.20) than on step faults (+0.09, interval including zero). The hybrid on the 7B model is ahead of the default rule (+0.09, [+0.05, +0.13] with traces resampled) but level with the best re-tuned rule (+0.02 [−0.02, +0.07]). C2 on Fable 5.1 is behind the rule (−0.07 [−0.17, −0.00] with traces resampled).

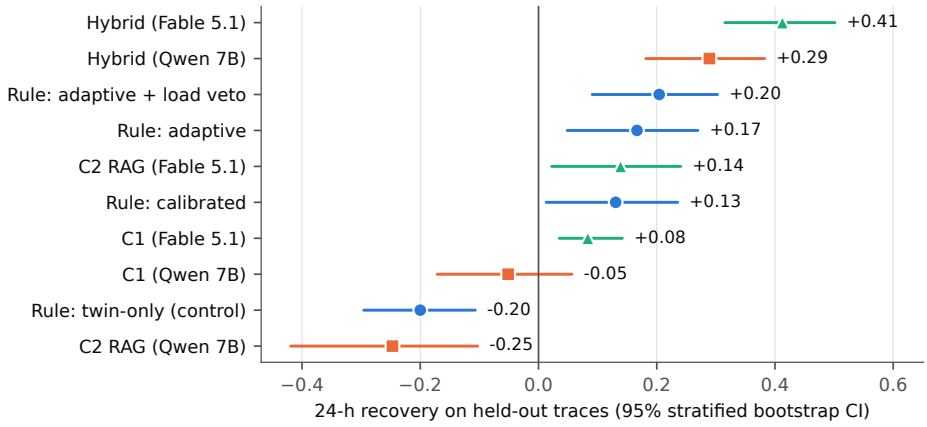


Figure 2: 24-h recovery on the held-out traces with the per-station detector. Circles: rule-based planners; squares: Qwen2.5 7B; triangles: Claude Fable 5.1.

7.5. What the vetoes rest on

The same hybrid on Fable 5.1 is the most conservative planner under the legacy detector and the best one under the per-station detector, with the same prompt. Its stated reasons barely change. Coded with an explicit keyword rule (*burst*, *line-wide*, *common cause*, *systemic*; rule and counts in the repository), 99% of its vetoes argue that the alarms come in bursts shared by many stations under the legacy detector (1 596 of the 1 607 calls with a veto), and 91% under the per-station detector (2 587 of 2 833); counted over distinct responses rather than calls, 97% (119 of 123) and 87% (249 of 285). Its vetoes of the faulty station cite this reason in 148 of 148 and 139 of 142 cases. The legacy detector raises an alarm *only* when more than 10% of the stations are flagged in the same batch, so under it every alarm is a shared burst and the argument vetoes nearly everything: the hybrid vetoed at least one station in 1 607 of its 1 812 calls. Under the per-station detector, a burst is one possible pattern among others, and the same argument separates line-wide disturbances from isolated faults: the hybrid vetoes in 2 833 of 4 348 calls and recovers twice as much as the rule. The hybrid's prompt names such bursts among the reasons to doubt the test; the 7B model, with the same prompt, uses this argument in fewer than 1% of its vetoes under either detector. The stated reasons alone cannot show how the model weighs evidence, but the contrast between the two Analyze layers, with planner and prompt fixed, shows that what the alarms encode decides whether the veto helps or hurts.

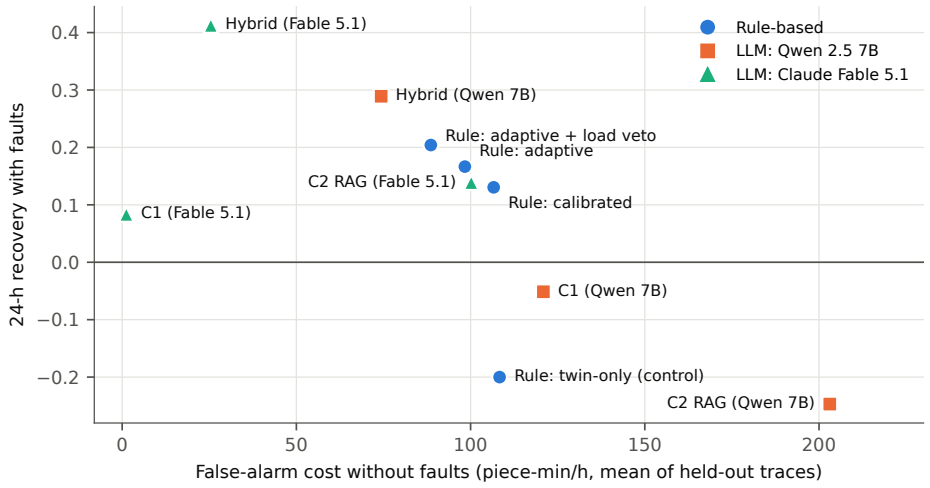


Figure 3: 24-h recovery with faults against false-alarm cost without faults, held-out traces, per-station detector.

7.6. Why the evaluation design matters

A first version of this study, circulated for review, injected every fault 600 min after the first event of a trace, let a repair clear a fault that began during it, and measured recovery over the whole trace. Under that design the legacy detector appeared to work: the adaptive rules recovered 52–54%, C2 on Fable 5.1 58% and the hybrid on Fable 5.1 24% (Table 5). A reviewer showed why. The fixed onset fell inside the detector's start-up burst of alarms and right after the 8-h warm-up of the planners' memory, when C2 repairs many stations at once; routine repairs then coincided with the fault or preceded it by minutes and, under the old repair rule, cleared it at no cost. Moving onsets out of the start-up period and letting repairs clear only active faults removed those gains and reversed the ranking of the two LLM designs. We report the first design here because the error is easy to make: a closed-loop controller evaluated at a single, fixed fault time inherits whatever happens to the line at that time.

8. Discussion

The Analyze layer decides. Closing the 2020 loop on its own detector recovers nothing within a day, whatever the planner: no planner differs from doing nothing by more than 0.07, because the detector gives too little and

Table 4: The hybrid planner on Claude Fable 5.1 against other planners, per-station detector, paired over the 216 held-out fault scenarios. Wilcoxon p-values Holm-adjusted over the rows of each block; r: matched-pairs rank-biserial correlation. Δ: difference in 24-h recovery (positive favours the hybrid on Fable 5.1), 95% bootstrap interval stratified by trace. Re-tuned rules: selected on the development trace, except the last row (best on the held-out traces). The breakdown by fault shape is exploratory.

Against	Better / worse	p (Holm)	r	Δ recovery [95% CI]
Adaptive + load veto	139 / 18	$< 10^{-15}$	0.78	+0.21 [+0.14, +0.28]
Adaptive rule	144 / 14	$< 10^{-15}$	0.81	+0.25 [+0.17, +0.32]
Calibrated rule	138 / 22	$< 10^{-15}$	0.77	+0.28 [+0.20, +0.37]
Hybrid, Qwen 7B	133 / 25	$< 10^{-12}$	0.68	+0.12 [+0.06, +0.19]
C2 (RAG), Fable 5.1	141 / 20	$< 10^{-15}$	0.82	+0.27 [+0.20, +0.35]
Against re-tuned rules				
Calibrated, re-tuned	116 / 37	$< 10^{-10}$	0.62	+0.15 [+0.08, +0.22]
Adaptive + load veto, re-tuned	143 / 18	$< 10^{-15}$	0.78	+0.23 [+0.16, +0.31]
Adaptive + load veto, best on held-out	128 / 31	$< 10^{-12}$	0.66	+0.13 [+0.06, +0.21]
Against adaptive + load veto, by fault shape				
step	42 / 7	$< 10^{-4}$	0.64	+0.09 [−0.02, +0.22]
intermittent	49 / 5	$< 10^{-6}$	0.85	+0.49 [+0.35, +0.64]
drift	48 / 6	$< 10^{-6}$	0.83	+0.20 [+0.13, +0.27]

too unspecific evidence about the faulty station. Replacing it with a per-station test on processing times, chosen on one development trace and kept fixed, moves the best rule from −0.02 to +0.20 and the best planner from +0.07 to +0.41. In MAPE-K terms, the information passed from Analyze to Plan is the first thing to get right.

Propose and veto. Given useful alarms, the LLM adds most as a veto over a statistical test, not as a triage on its own. The test supplies a candidate only when a station's alarm rate departs from its own history; the model, with the retrieved statistics, rejects candidates that a line-wide disturbance, a reroute or an ineffective past repair explains. This division of labour keeps the decision anchored in a transparent test and uses the model where it has an advantage: weighing several pieces of context that a rule would have to enumerate. On their own, both LLM planners do worse than the rule; the frontier model without retrieved knowledge mostly declines to act.

Capability matters inside the right architecture. The same hybrid recovers +0.41 on Fable 5.1 and +0.29 on the 7B model, which only ties the best re-tuned rule. This is a contrast between two systems, which differ in size, quantisation, provider and inference settings, not a measured capability curve.

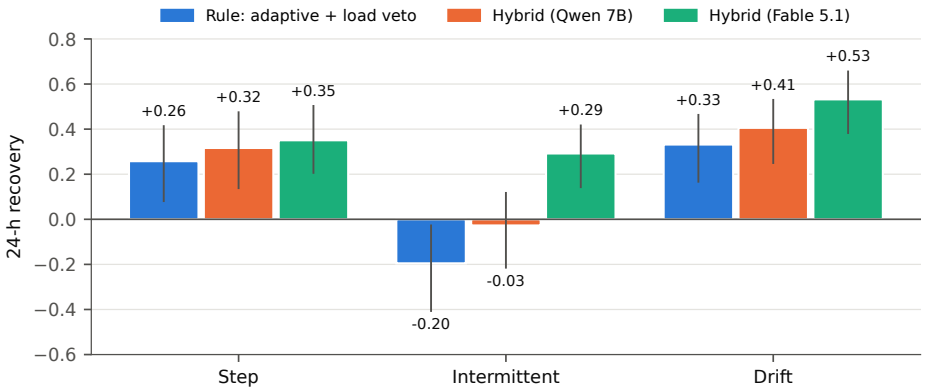


Figure 4: 24-h recovery by fault shape on the held-out traces, per-station detector (72 scenarios per shape). Whiskers: 95% bootstrap interval stratified by trace.

Table 5: The same planners under the first evaluation design (v1: fault at 600 min, a repair clears faults that begin during it, recovery over the whole trace, 72 scenarios) and the present one (v2: three random onsets after the start-up period, a repair clears only active faults, 24-h recovery, 216 scenarios). Held-out traces.

Planner	v1, legacy	v2, legacy	v2, per-station
Adaptive + load veto	+0.54	−0.02	+0.20
C2 (RAG), Qwen 7B	+0.20	−0.15	−0.25
Hybrid, Qwen 7B	+0.41	−0.04	+0.29
C2 (RAG), Fable 5.1	+0.58	+0.07	+0.14
Hybrid, Fable 5.1	+0.24	+0.03	+0.41

Implications for practice. Within the limits of a study with synthetic faults, four points transfer to automation architects and maintenance engineers. First, connecting threshold alarms directly to maintenance orders is harmful: the naive planner loses against doing nothing under both detectors, because false repairs accumulate; alarms must be rationalised before they drive actions, by analogy with what alarm-management practice requires for operators [13]. Second, a detector should compare each station with its own recent behaviour on the part of the signal a fault affects, here processing times rather than storage-dominated transitions. Third, a rate-change test is a cheap and transparent proposal layer and the reference any learned or LLM-based controller should be compared with. Fourth, an LLM can sit on top of such a test as an advisory veto; its non-determinism and its sensitivity to how alarms are presented argue for keeping every action behind a

symbolic validator, as here, and, in a plant, behind human approval.

Development sets mislead. The per-station detector and the re-tuned rules were chosen on the development trace, where the adaptive rule recovers +0.84; on the held-out traces it recovers +0.20, and the configurations re-tuned there help one rule and not the others. Under the first design, three conclusions drawn on the development trace were also wrong. A single trace is too small to select designs on, and LLM-based controllers should be evaluated on data not used to write their prompts, with fault onsets that do not coincide with the controller's start-up.

Threats to validity. Construct: faults are synthetic, and the effect model is additive and trace-driven, without queueing, so every recovery is a property of the model, not a measured property of the Bosch line. Its unverifiable parameters were swept one at a time for the rules (Table 2): mild faults, long repairs, full congestion and stop-mode maintenance remove or reverse the benefit of acting, so every recovery we report holds only for severe faults, short repairs and backup capacity during repairs. No sensitivity analysis was run for the LLM planners. Recovery is measured in production time within 24 h of the fault, without economic weights; the 24-h window can fall partly on hours without production, because the traces are recorded in batches. Twins are structural, not verified equipment equivalents. *Internal*: each frontier-model decision is a single stored sample at low effort, and most calls reuse a stored response; the prompts are part of the treatment, and the hybrid's prompt contains the burst cue its vetoes cite; the 7B model is 4-bit quantised. The per-station detector and the evaluation design were introduced after a first round of review had seen results under the first design; both were fixed on the development trace, and the change is recorded in the repository before any held-out run with them. The frontier model may have seen our 2020 papers, which name the Bosch line, but the faults are synthetic and not described anywhere it could have seen. *External*: one production line, one dataset; two language models, with no intermediate sizes. *Statistical*: scenarios on the same trace are not independent; three held-out traces are few clusters, which the trace-level resampling addresses only partly.

9. Conclusion

We closed the autonomic loop of a self-coordinated manufacturing architecture that, as published, only detected. On its own detector, no planner

recovers delay: the detector raises line-wide bursts of alarms and reports a faulty station rarely. With a per-station detector on processing times, a statistical change test recovers about a fifth of the delay within a day, and a frontier LLM with retrieved operational knowledge that vetoes the test's candidates recovers about two fifths, ahead of every rule configuration we tried. Used on its own, the same model does not beat the test. An earlier evaluation, with a fixed fault time, had suggested the opposite ranking. Future work will validate the effect model against plant data with real capacities and repair times, run the hybrid without the burst cue and with repeated samples, add intermediate model sizes, and extend the sensitivity analysis to the LLM planners.

Appendix A. Prompts

System prompt v3 of C1, as sent to both models (exported from the code):

```
You are the Plan phase of an autonomic controller for a manufacturing line
     (the Bosch production line: about 50 stations on 4 sub-lines).

How alarms work. The Analyze phase checks each station separately. It
    raises an alarm listing the stations whose recent processing times (
    transitions of up to 60 minutes leaving the station) exceed the
    station's own recent history more often than chance would explain. In
    normal operation this detector also raises false alarms: a station
    named in an alarm may be perfectly healthy.

What you can do for each flagged station:
- "none": leave it alone;
- "maintain": repair it for 60 minutes. While it is being repaired a
    slower backup handles its work. Repairing a degraded station fixes it.
     Repairing a healthy station fixes nothing and only slows production;
- "reroute_and_maintain": only for a station with a free twin. The twin
    takes all of the station's load during the repair (the twin slows down
     because of the extra load), and the station is repaired.

Objective: minimise total production delay. Acting on a healthy station
    wastes production. Ignoring a truly degraded station lets its delay
    accumulate for as long as it stays degraded.

For EVERY station listed, first give a verdict, then an action:
- "degraded": the evidence shows the station itself is not working
    properly;
- "normal": the station is probably healthy and the alarm is a false alarm
    ;
```

```
- "self_inflicted": the station is alarming because of load you rerouted
    onto it;
- "insufficient_evidence": you cannot tell.
Only a "degraded" station may receive an action other than "none".

Answer only with JSON matching the schema. Keep "reason" under 30 words.
```

C2 appends:

```
You also receive RETRIEVED KNOWLEDGE for each station, computed from your
    operational log:
- how many alarms the station raised recently, and how many its normal (
    fault-free) alarm rate would predict over the same period;
- whether you are currently pushing extra load onto it;
- what happened to its alarms after you repaired it before;
- its recent log records.
How to read it:
- many more alarms than expected, sustained over the last 24 h, points to
    "degraded";
- about as many alarms as expected points to "normal";
- a station receiving load you rerouted onto it is "self_inflicted":
    repairing it does not help;
- if alarms did not drop after a previous repair, the alarms were probably
     not caused by a fault;
- with too little history, prefer "insufficient_evidence".
```

The hybrid appends, after C2's text:

```
In this mode a statistical change detector has already screened the alarms
    . The stations listed were flagged because their recent alarm count is
     improbably high compared with their own recent history. The detector
    is usually right, but it cannot see context: load you rerouted onto a
    station, a previous repair that did not reduce its alarms, a burst of
    alarms shared by many stations at once. Give "degraded" unless the
    retrieved knowledge gives a concrete reason to doubt the detector;
    then give the verdict that reason supports.
```

Prompt v2, used with the legacy detector, replaces the first sentence of the second paragraph of the C1 prompt (from "How alarms work" to "explain.") with:

```
How alarms work. The Analyze phase raises an alarm listing stations whose
    recent transition times exceeded 1.3 times their reference time within
     the same batch of events.
```

The user message lists the time, the stations to judge with the state of their twins, the stations under repair and the active reroutes; for C2 and the hybrid it adds, per station, the retrieved knowledge described in Section 5. The JSON schema of the answer is in the repository.

Data and code availability

The headless simulator, planners, prompts, development log, every model decision with its stated reason, the results and the scripts that produce every table and figure are available at `https://github.com/mbsanchez/ssacsimulator`. The simulator uses the Python standard library only; the statistics and figures additionally use SciPy and matplotlib. The Bosch dataset is publicly available [16]; the files derived from it (process model and event traces) are not redistributed, and the repository gives the scripts that produce them and their checksums.

CRediT authorship contribution statement

Manuel Sánchez: Conceptualization, Methodology, Investigation, Visualization, Writing – original draft. **Ernesto Exposito:** Supervision, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used Claude (Anthropic) to assist in developing the simulation and analysis software, to draft and revise the text of the manuscript, and as an adversarial agent acting as a peer reviewer to identify weaknesses and suggest improvements. The software was reviewed by the first author. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article. The use of Qwen2.5 and Claude Fable 5.1 as planners is part of the method studied in this paper (Section 5) and is unrelated to the preparation of the manuscript.

References

[1] J. O. Kephart, D. M. Chess, The vision of autonomic computing, Computer 36 (1) (2003) 41–50. `doi:10.1109/MC.2003.1160055`.

[2] D. Weyns, An Introduction to Self-adaptive Systems: A Contemporary Software Engineering Perspective, Wiley-IEEE Press, 2021. doi:10.1002/9781119574910.

[3] M. Sánchez, E. Exposito, J. Aguilar, Implementing self-* autonomic properties in self-coordinated manufacturing processes for the Industry 4.0 context, Computers in Industry 121 (2020) 103247. doi:10.1016/j.compind.2020.103247.

[4] M. Sánchez, E. Exposito, J. Aguilar, Autonomic computing in manufacturing process coordination in Industry 4.0 context, Journal of Industrial Information Integration 19 (2020) 100159. doi:10.1016/j.jii.2020.100159.

[5] Y. Xia, M. Shenoy, N. Jazdi, M. Weyrich, Towards autonomous system: flexible modular production system enhanced with large language model agents, in: 2023 IEEE 28th International Conference on Emerging Technologies and Factory Automation (ETFA), 2023, pp. 1–8. doi:10.1109/ETFA54631.2023.10275362.

[6] M. Li, Q. Zhou, W. Li, T. Qu, M. Yang, P. Jiang, A4PS: Agentic AI-assisted advanced planning and scheduling with large language models for smart manufacturing, Journal of Manufacturing Systems 85 (2026) 207–226. doi:10.1016/j.jmsy.2026.01.003.

[7] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, D. Kiela, Retrieval-augmented generation for knowledge-intensive NLP tasks, in: Advances in Neural Information Processing Systems, Vol. 33, 2020, pp. 9459–9474.

[8] J. Li, M. Zhang, N. Li, D. Weyns, Z. Jin, K. Tei, Generative AI for self-adaptive systems: State of the art and research roadmap, ACM Transactions on Autonomous and Adaptive Systems 19 (3) (2024) 13:1–13:60. doi:10.1145/3686803.

[9] J. Li, M. Zhang, N. Li, D. Weyns, Z. Jin, K. Tei, Exploring the potential of large language models in self-adaptive systems, in: Proceedings of the 19th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS), 2024, pp. 77–83. doi:10.1145/3643915.3644088.

[10] N. Nascimento, P. Alencar, D. Cowan, Self-adaptive large language model (LLM)-based multiagent systems, in: 2023 IEEE International Conference on Autonomic Computing and Self-Organizing Systems Companion (ACSOS-C), 2023, pp. 104–109. doi:10.1109/ACSOS-C58168.2023.00048.

[11] L. Alves, G. L. B. De Oliveira, M. Chagas, A. Bandeira, P. H. Maia, C. Rocha, MAPER: Extending MAPE-K with LLM-based reasoning to manage unanticipated situations in self-adaptive systems, in: Proceedings of the 21st International Conference on Software Engineering for Adaptive and Self-Managing Systems (SEAMS), 2026, pp. 239–250. doi:10.1145/3788550.3794886.

[12] M. Sánchez, E. Exposito, J. Aguilar, Industry 4.0: survey from a system integration perspective, International Journal of Computer Integrated Manufacturing 33 (10–11) (2020) 1017–1041. doi:10.1080/0951192X.2020.1775295.

[13] International Society of Automation, ANSI/ISA-18.2-2016: Management of alarm systems for the process industries, Tech. rep., International Society of Automation, Research Triangle Park, NC (2016).

[14] M. Basseville, I. V. Nikiforov, Detection of Abrupt Changes: Theory and Application, Prentice Hall, 1993.

[15] N. D. Thuan, L. H. Anh, Large language models for fault diagnosis in smart manufacturing: A comprehensive review, Intelligent Systems with Applications 31 (2026) 200714. doi:10.1016/j.iswa.2026.200714.

[16] Bosch, Bosch production line performance, Kaggle competition dataset, https://www.kaggle.com/c/bosch-production-line-performance (2016).

[17] Qwen Team, Qwen2.5 technical report, arXiv:2412.15115 (2024).

[18] Ollama, Ollama: run large language models locally, https://ollama.com, version 0.35.1 (2026).

[19] Anthropic, Claude agent sdk, https://code.claude.com/docs/en/agent-sdk, python package claude-agent-sdk 0.2.163 (2026).

[20] B. Efron, R. J. Tibshirani, An Introduction to the Bootstrap, Chapman & Hall, 1993.

[21] F. Wilcoxon, Individual comparisons by ranking methods, Biometrics Bulletin 1 (6) (1945) 80–83.

[22] S. Holm, A simple sequentially rejective multiple test procedure, Scandinavian Journal of Statistics 6 (2) (1979) 65–70.